

Understanding the Master Theorem for solving Recurrence Relations

1 What is the Master Theorem?

The Master Theorem is a tool to analyze the time complexity of divide-and-conquer algorithms. It solves recurrence relations of the form:

$$T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

where:

- $a \geq 1$ is the number of subproblems
- $b > 1$ is the factor by which the input size is divided
- $f(n)$ is the cost of dividing and combining the subproblems

2 Step-by-Step Approach

Step 1: Identify a , b , and $f(n)$.

Step 2: Compute $E = \log_b^a$.

Step 3: Compare $f(n)$ with n^E .

Step 4: Use one of the following cases:

Case-1 If $f(n) = O(n^{E-\varepsilon})$ for some $\varepsilon > 0$, then $T(n) = \Theta(n^E)$

Case-2 If $f(n) = \Theta(n^E)$, then $T(n) = \Theta(n^E \log n)$

Case-3 If $f(n) = \Omega(n^{E+\varepsilon})$ for some $\varepsilon > 0$, and the regularity condition holds: $a \cdot f\left(\frac{n}{b}\right) \leq c \cdot f(n)$ for some $c < 1$ and large n , then $T(n) = \Theta(f(n))$

3 Example 1: Merge Sort

Recurrence: $T(n) = 2T(n/2) + \Theta(n)$

- $a = 2$, $b = 2$, $f(n) = \Theta(n)$

- $\log_b^a = \log_2^2 = 1$
- $f(n) = \Theta(n^1)$, so it matches Case 2

Answer: $T(n) = \Theta(n \log n)$

4 Example 2: Binary Search

Recurrence: $T(n) = T(n/2) + \Theta(1)$

- $a = 1, b = 2, f(n) = \Theta(1)$
- $\log_b^a = \log_2^1 = 0$
- $f(n) = \Theta(n^0)$, so it matches Case 2

Answer: $T(n) = \Theta(\log n)$

5 Example 3: Dominant $f(n)$

Recurrence: $T(n) = 2T(n/2) + \Theta(n^2)$

- $a = 2, b = 2, f(n) = \Theta(n^2)$
- $\log_b^a = \log_2^2 = 1$
- $f(n) = \Omega(n^{1+\varepsilon})$ with $\varepsilon = 1$
- Check regularity: $2 \cdot (n/2)^2 = n^2/2 < c \cdot n^2$ for $c = 1$

Answer: $T(n) = \Theta(n^2)$ (Case 3)

6 When Not to Use the Master Theorem

- When the recurrence is not in the right form, e.g., $T(n) = T(n-1) + n$
- When subproblem sizes are not equal or not divided by a constant factor
- When $f(n)$ is not a polynomial-type function (like logarithmic or exponential)